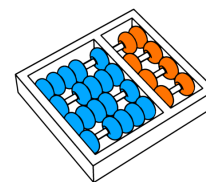




Relatório de desenvolvimento das atividades
Instituto de Computação - Universidade Estadual de Campinas
Allan M. de Souza, Rafael O. Jarczewski



Nome: Randerson Araujo de Lemos
RA: 103897

1 Introdução

Este documento apresenta a metodologia e os resultados da configuração da arquitetura de rede corporativa proposta no Desafio Corporativo (Trabalho 4 - MC833). O objetivo central foi estabelecer a conectividade completa entre múltiplos polos e a gerência central, garantindo o acesso à internet via NAT, ao mesmo tempo em que se implementava uma robusta política de segurança baseada em firewall stateful. O projeto focou no isolamento obrigatório entre as filiais e na proteção de recursos críticos (Banco de Dados e DNS). O processo de desenvolvimento foi dividido em cinco fases sequenciais, detalhadas a seguir:

O desenvolvimento seguiu uma progressão lógica de cinco etapas, detalhadas nas seções a seguir:

1. **Atribuição de IPs aos Roteadores:** Aborda a definição do esquema de sub-redes (`192.168.100.0/24` com máscara `/26`) e a atribuição de IPs estáticos aos roteadores, incluindo a solução para o problema de mapeamento não-determinístico das interfaces do Docker.
2. **Atribuição de IPs aos Hosts:** Detalha a alocação de IPs para todos os dispositivos finais da rede e a configuração inicial de seus respectivos gateways locais.
3. **Roteamento Estático:** Explica a configuração das rotas padrão (`default`) nos hosts e as rotas estáticas nos roteadores para permitir a comunicação entre todas as sub-redes corporativas.
4. **NAT (Network Address Translation):** Descreve a implementação da regra `MASQUERADE` no roteador de borda (`r4_edge`) para viabilizar o acesso dos hosts internos à internet pública.
5. **Regras de Firewall:** Apresenta a aplicação das políticas de segurança com `iptables` em cada roteador, focando no isolamento das filiais e na proteção dos servidores, além de bloquear conexões externas não solicitadas.
- 6.

A documentação técnica dos comandos de configuração (`run.sh`) e dos testes de validação (`test.sh`) está estruturada nos scripts de shell, localizados no diretório principal `scripts/`. O projeto está dividido em cinco subpastas sequenciais, onde cada par de scripts corresponde a uma fase do trabalho, conforme detalhado nas seções subsequentes:

7. **scripts/01_routers/:** Atribui IPs estáticos aos roteadores, lidando com o mapeamento não-determinístico das interfaces.
8. **scripts/02_hosts/:** Configura IPs estáticos nos dispositivos finais (hosts).
9. **scripts/03_routing/:** Implementa o roteamento estático e os *gateways default* para permitir a comunicação entre todas as sub-redes.
10. **scripts/04_nat/:** Configura a regra `MASQUERADE` no roteador de borda (`r4_edge`) para o acesso à internet.
11. **scripts/05_firewall/:** Aplica as regras de segurança com `iptables` para isolamento e proteção da rede interna.

O conteúdo de todos esses arquivos também pode ser consultado na seção de anexo no final deste documento.

2 Atribuição de IPs aos Roteadores

A fase inicial do projeto concentrou-se na atribuição de endereços IP estáticos às interfaces dos quatro roteadores da rede. Esta etapa é fundamental, pois a definição dos gateways padrão para todos os demais dispositivos da rede depende da correta configuração dos roteadores. Sem esta base, o tráfego não seria capaz de transitar entre as sub-redes.

Configurações

A rede usa o bloco `192.168.100.0/24` (Classe C) dividido em 4 sub-redes com máscara `/26`, gerando blocos de 64 endereços (62 utilizáveis). O primeiro endereço utilizável de cada sub-rede é reservado como gateway.

Roteador	Sub-rede	IP atribuído	Papel
r1_hq	net_servico	192.168.100.1/26	Identidade no backbone
r1_hq	net_gerencia	192.168.100.193/26	Gateway da gerência
r2_br1	net_servico	192.168.100.2/26	Identidade no backbone
r2_br1	net_polo1	192.168.100.65/26	Gateway da Filial 1
r3_br2	net_servico	192.168.100.3/26	Identidade no backbone
r3_br2	net_polo2	192.168.100.129/26	Gateway da Filial 2
r4_edge	net_servico	192.168.100.4/26	Identidade no backbone
r4_edge	net_internet	203.0.113.1/24	IP público (NAT)

Implementação

As configurações foram implementadas no script `scripts/01_routers/run.sh`. O script é responsável por identificar quais interfaces de rede estão associadas a cada sub-rede, remover os IPs temporários atribuídos pelo Docker e atribuir os IPs estáticos corretos.

Problema das interfaces. Cada roteador possui duas interfaces (`eth0` e `eth1`), uma para cada rede. O principal desafio técnico desta fase foi a não-determinação das interfaces de rede pelo Docker. Como o mapeamento das interfaces às redes do Docker é inconsistente a cada inicialização, houve um risco de que o endereço IP de uma sub-rede ficasse associado à interface errada (por exemplo, o IP da Filial 1 na interface do backbone). Tal desalinhamento impediria a comunicação, visto que a conectividade na Camada 2 (bridge) exige que o IP esteja na interface correta. Tentativas de forçar o mapeamento via `priority` no `docker-compose.yml` foram ineficazes, exigindo uma abordagem dinâmica.

Solução: Detecção em Tempo de Execução. Para contornar a imprevisibilidade do Docker, implementamos uma solução de detecção em *runtime*. O Docker atribui um IP

temporário (172.x.x.x) a cada interface, servindo como um identificador único do *bridge* ao qual a interface está conectada. O script aproveita essa característica, realizando uma inspeção da rede Docker (`docker network inspect`) para correlacionar o IP temporário com a rede lógica desejada. Em seguida, localiza qual interface dentro do container possui esse IP, assegurando que o endereço estático seja aplicado à interface correta, independentemente do nome (`eth0` ou `eth1`) atribuído pelo sistema.

Shell

```
# Obtém o IP temporário que o Docker atribuiu ao r2_br1 na
net_polo1
docker network inspect student-environment_net_polo1 \
| grep -A4 '"Name": "r2_br1"' \
| grep -oE '172\.[0-9]+\.[0-9]+\.[0-9]+'
# Localiza qual interface possui esse IP dentro do container
docker exec r2_br1 ip addr show \
| awk -v ip="172.x.x.x" '
/^[0-9]+:/{ split($2, a, "@"); iface = a[1]; gsub(/:$/, "",
iface) }
/inet / { if ($2 ~ ip) print iface }
'
```

Com a interface correta identificada, os dois comandos abaixo são aplicados para cada rede de cada roteador:

Shell

```
# Remove o IP temporário do Docker para evitar endereços
duplicados na interface
docker exec r2_br1 ip addr flush dev eth1
# Atribui o IP estático à interface correta
docker exec r2_br1 ip addr add 192.168.100.65/26 dev eth1
```

O `ip addr flush` é necessário porque, sem ele, a interface ficaria com dois IPs — o temporário 172.x.x.x e o estático — gerando rotas conectadas duplicadas na tabela do kernel. O `ip addr add` atribui o endereço estático e, como efeito automático, o kernel cria uma rota conectada para toda a sub-rede naquela interface, por exemplo:

None

```
192.168.100.64/26    dev    eth1    proto    kernel    scope    link    src
192.168.100.65
```

Essa rota indica que qualquer pacote destinado à Filial 1 sai diretamente pela interface conectada ao bridge da Filial 1, sem precisar de um roteador intermediário.

Verificação

A verificação foi implementada no script `scripts/01_routers/test.sh` e está dividida em três níveis. O **Nível 1** confirma que cada IP estático foi atribuído ao container, buscando o endereço na saída do `ip addr show` e reportando em qual interface ele foi parado. O **Nível 2** verifica se todos os roteadores conseguem se pingar mutuamente pelo backbone `net_servico`. Como todos os IPs do backbone (.1, .2, .3, .4) pertencem ao mesmo /26, a comunicação deve ocorrer diretamente no bridge, sem roteamento. Este é o teste definitivo: o Nível 1 pode passar mesmo com a interface errada (o IP existe no container), mas o Nível 2 falha se o IP estiver no bridge errado, pois os demais roteadores não o enxergam. O **Nível 3** exibe a tabela de roteamento de cada roteador (`ip route`) para confirmar que o kernel criou as rotas conectadas esperadas após a atribuição dos IPs.

As tabelas abaixo mostram o estado do kernel após a execução do script. Neste ponto, cada roteador possui apenas **rotas conectadas** — criadas automaticamente pelo kernel ao receber um IP. Rotas para sub-redes remotas e rota padrão são adicionadas na etapa seguinte (roteamento estático).

Note que os nomes das interfaces (`eth0/eth1`) variam conforme o que o Docker atribuiu nessa execução específica.

r1_hq

None

```
192.168.100.0/26 dev eth1 proto kernel scope link src
192.168.100.1
192.168.100.192/26 dev eth0 proto kernel scope link src
192.168.100.193
```

r2_br1

None

```
192.168.100.0/26 dev eth0 proto kernel scope link src
192.168.100.2
192.168.100.64/26 dev eth1 proto kernel scope link src
192.168.100.65
```

r3_br2

None

```
192.168.100.0/26 dev eth1 proto kernel scope link src
192.168.100.3
192.168.100.128/26 dev eth0 proto kernel scope link src
192.168.100.129
```

r4_edge

None

```
192.168.100.0/26 dev eth0 proto kernel scope link src
192.168.100.4
203.0.113.0/24 dev eth1 proto kernel scope link src 203.0.113.1
```

Cada linha segue o formato `<sub-rede> dev <interface> src <IP-do-roteador>`. A ausência de `via` indica rota direta — o roteador entrega o pacote sem passar por outro salto.

3 Atribuição de IPs aos Hosts

Com a infraestrutura de roteadores devidamente estabelecida, a segunda etapa focou na alocação de endereços IP estáticos para todos os dispositivos finais, incluindo servidores corporativos, estações de trabalho e clientes da internet simulada. A atribuição de um IP é essencial para que o host se torne alcançável e possa iniciar ou responder ao tráfego na rede.

Configurações

Diferente dos roteadores, os hosts possuem apenas uma interface (`eth0`) e recebem um único IP dentro da sua sub-rede. O gateway de cada host é o IP do roteador responsável por aquela sub-rede, configurado no passo anterior.

Dispositivo	IP atribuído	Sub-rede	Gateway
srv_dns	192.168.100.10/26	net_servico	192.168.100.1 (r1_hq)
srv_web	192.168.100.11/26	net_servico	192.168.100.1 (r1_hq)
srv_db	192.168.100.12/26	net_servico	192.168.100.1 (r1_hq)
pc1_br1	192.168.100.66/26	net_polo1	192.168.100.65 (r2_br1)
pc2_br1	192.168.100.67/26	net_polo1	192.168.100.65 (r2_br1)
pc3_br1	192.168.100.68/26	net_polo1	192.168.100.65 (r2_br1)
pc1_br2	192.168.100.130/26	net_polo2	192.168.100.129 (r3_br2)
pc2_br2	192.168.100.131/26	net_polo2	192.168.100.129 (r3_br2)
pc3_br2	192.168.100.132/26	net_polo2	192.168.100.129 (r3_br2)
admin_pc	192.168.100.194/26	net_gerencia	192.168.100.193 (r1_hq)
srv_public_web	203.0.113.10/24	net_internet	203.0.113.1 (r4_edge)
ext_client	203.0.113.20/24	net_internet	203.0.113.1 (r4_edge)

Implementação

As configurações foram implementadas no script `scripts/02_hosts/run.sh`. O padrão aplicado a cada host é idêntico ao dos roteadores: remover o IP temporário do Docker e atribuir o IP estático.

Shell

```
# Remove o IP temporário atribuído pelo Docker
docker exec pc1_br1 ip addr flush dev eth0

# Atribui o IP estático dentro da sub-rede de Polo 1
docker exec pc1_br1 ip addr add 192.168.100.66/26 dev eth0
```

Como os hosts utilizam apenas uma interface (`eth0`), o desafio do mapeamento não-determinístico presente nos roteadores foi evitado. O processo de implementação seguiu um padrão claro: o comando `ip addr flush` removeu o IP temporário (`172.x.x.x`) atribuído pelo Docker, e o `ip addr add` configurou o IP estático com a máscara `/26` definida. A aplicação desse endereço estático resultou na criação automática de uma rota conectada pelo kernel (ex: `192.168.100.64/26 dev eth0 proto kernel...`), garantindo a comunicação direta (Camada 2) entre hosts da mesma sub-rede (como `pc1_br1` e `pc2_br1`). No entanto, o isolamento entre sub-redes persiste, pois a rota padrão (default gateway) ainda não foi configurada, o que será abordado na próxima fase.

None

```
192.168.100.64/26 dev eth0 proto kernel scope link src
192.168.100.66
```

Essa rota significa que `pc1_br1` consegue alcançar `pc2_br1` (.67) e `pc3_br1` (.68) diretamente — mesma sub-rede, mesmo bridge, sem passar pelo roteador. No entanto, nenhum host consegue alcançar dispositivos em outras sub-redes ainda, pois não há rota padrão configurada. Isso é resolvido no próximo passo.

Verificação

A verificação foi implementada no script `scripts/02_hosts/test.sh`. Para cada dispositivo, o teste busca o IP esperado na saída do `ip addr show`:

Shell

```
docker exec pc1_br1 ip addr show | grep 192.168.100.66
# Output: inet 192.168.100.66/26 scope global eth0
```

Se o IP estiver presente, o teste passa. Em seguida, o script exibe a tabela de roteamento de cada host para confirmar que apenas a rota conectada existe — sem rota padrão, o que é o estado correto ao final deste passo.

Ao final deste passo, cada host possui apenas uma rota — a rota conectada criada automaticamente pelo kernel. Não há rota padrão (`default`) ainda.

Servidores corporativos (net_servico)

None

```
srv_dns   →   192.168.100.0/26   dev   eth0   proto   kernel   src
192.168.100.10
srv_web   →   192.168.100.0/26   dev   eth0   proto   kernel   src
192.168.100.11
srv_db    →   192.168.100.0/26   dev   eth0   proto   kernel   src
192.168.100.12
```

Clientes da Filial 1 (net_polo1)

None

```
pc1_br1   →   192.168.100.64/26   dev   eth0   proto   kernel   src
192.168.100.66
pc2_br1   →   192.168.100.64/26   dev   eth0   proto   kernel   src
192.168.100.67
pc3_br1   →   192.168.100.64/26   dev   eth0   proto   kernel   src
192.168.100.68
```

Clientes da Filial 2 (net_polo2)

None

```
pc1_br2   →   192.168.100.128/26   dev   eth0   proto   kernel   src
192.168.100.130
pc2_br2   →   192.168.100.128/26   dev   eth0   proto   kernel   src
192.168.100.131
pc3_br2   →   192.168.100.128/26   dev   eth0   proto   kernel   src
192.168.100.132
```

Gerência (net_gerencia)

None

```
admin_pc  →   192.168.100.192/26   dev   eth0   proto   kernel   src
192.168.100.194
```

Internet simulada (net_internet)

None

```
srv_public_web → 203.0.113.0/24 dev eth0 proto kernel src 203.0.113.10
ext_client → 203.0.113.0/24 dev eth0 proto kernel src 203.0.113.20
```

4 Roteamento Estático

A etapa de Roteamento Estático foi crucial para conferir inteligência à rede. Após a atribuição dos IPs, os dispositivos estavam limitados a se comunicar apenas dentro de suas respectivas sub-redes (rotas conectadas). Um pacote destinado a uma sub-rede remota seria simplesmente descartado pelo kernel. O objetivo desta fase foi, portanto, instruir cada host e roteador sobre o caminho correto (*next-hop*) para alcançar destinos fora de sua rede local.

Configurações

Existem dois tipos de configuração: **gateway padrão** para os hosts e **rotas estáticas** para os roteadores.

Gateway padrão dos hosts (cada host envia todo tráfego desconhecido ao roteador da sua sub-rede):

Dispositivo	Gateway padrão
srv_dns, srv_web, srv_db	192.168.100.4 (r4_edge)
pc1/2/3_br1	192.168.100.65 (r2_br1)
pc1/2/3_br2	192.168.100.129 (r3_br2)
admin_pc	192.168.100.193 (r1_hq)
srv_public_web, ext_client	203.0.113.1 (r4_edge)

Por que os servidores usam r4_edge e não r1_hq como gateway? O gateway padrão é essencial apenas para o tráfego que precisa sair da sub-rede `net_servico` (192.168.100.0/26), ou seja, para a internet (203.0.113.0/24). Visto que o `r4_edge` é o roteador de borda e está diretamente conectado ao mesmo *backbone* que os servidores, configurá-lo como gateway principal otimiza o caminho. Essa escolha é estratégica e evita um salto redundante (via `r1_hq`), resultando em maior eficiência no tráfego de saída para a internet.

O gateway padrão só é necessário para tráfego destinado **fora** do /26, que na prática significa a internet (203.0.113.0/24). Como o `r4_edge` é o gateway da internet e está diretamente alcançável no mesmo backbone, apontá-lo como gateway evita um salto desnecessário:

None

```
# gateway = r1_hq: srv_dns → r1_hq → r4_edge → internet (2 saltos)
# gateway = r4_edge: srv_dns → r4_edge → internet (1 salto)
```

Usar `r1_hq` funcionaria, mas adicionaria um salto redundante em todo pacote destinado à internet.

Rotas estáticas dos roteadores (cada roteador já conhece as sub-redes às quais está diretamente conectado, precisando apenas aprender as rotas remotas, usando os IPs do backbone como next-hop):

Roteador	Sub-rede de destino	Next-hop
r1_hq	192.168.100.64/26 (Polo 1)	192.168.100.2 (r2_br1)
r1_hq	192.168.100.128/26 (Polo 2)	192.168.100.3 (r3_br2)
r1_hq	default	192.168.100.4 (r4_edge)
r2_br1	192.168.100.128/26 (Polo 2)	192.168.100.3 (r3_br2)
r2_br1	192.168.100.192/26 (Gerência)	192.168.100.1 (r1_hq)
r2_br1	default	192.168.100.4 (r4_edge)
r3_br2	192.168.100.64/26 (Polo 1)	192.168.100.2 (r2_br1)
r3_br2	192.168.100.192/26 (Gerência)	192.168.100.1 (r1_hq)
r3_br2	default	192.168.100.4 (r4_edge)
r4_edge	192.168.100.64/26 (Polo 1)	192.168.100.2 (r2_br1)
r4_edge	192.168.100.128/26 (Polo 2)	192.168.100.3 (r3_br2)
r4_edge	192.168.100.192/26 (Gerência)	192.168.100.1 (r1_hq)

O `r4_edge` não recebe rota padrão — ele é o gateway para a internet. Mas recebe rotas de retorno para todas as sub-redes internas, necessárias para que os pacotes de resposta do NAT cheguem ao host correto.

Por que `r1_hq` não aparece na tabela com rota para `net_gerencia`? É importante notar que o `r1_hq` não necessita de rotas estáticas para a sub-rede de Gerência. Isso ocorre porque o `r1_hq` está diretamente conectado à `net_gerencia`, e o kernel Linux cria automaticamente uma **rota conectada** assim que o IP é atribuído (Passo 1). A tabela de rotas estáticas serve apenas para mapear destinos que não estão diretamente acessíveis.

None

```
192.168.100.192/26 dev eth0 proto kernel ← criada automaticamente
```

Todos os roteadores conseguem alcançar a gerência — o caminho de cada um é:

Roteador	Como alcança net_gerencia
r1_hq	Conectado diretamente — entrega o pacote na própria interface
r2_br1	Rota estática → via 192.168.100.1 (r1_hq)
r3_br2	Rota estática → via 192.168.100.1 (r1_hq)
r4_edge	Rota estática → via 192.168.100.1 (r1_hq)

Por que os roteadores precisam de rotas estáticas? O emprego de rotas estáticas nos roteadores é indispensável para garantir a comunicação inter-sub-redes. Sem elas, um roteador como o `r2_br1`, que está conectado apenas ao Polo 1 e ao backbone, descartaria o tráfego destinado a redes desconhecidas (como a Gerência ou o Polo 2). As rotas estáticas resolvem esse problema de decisão, fornecendo o caminho exato (*next-hop*) para as sub-redes remotas.

Por que os next-hops são sempre IPs do backbone? Todos os quatro roteadores compartilham `net_servico` (192.168.100.0/26), o que significa que cada um alcança os demais diretamente — sem intermediários. Por isso os next-hops são sempre IPs do backbone: `r2_br1` não precisa conhecer o caminho completo até `net_gerencia`, apenas sabe que o próximo salto é `r1_hq` (192.168.100.1), que está diretamente acessível na mesma /26. O resto é responsabilidade de `r1_hq`.

Por que o r4_edge não tem rota padrão? Os demais roteadores apontam tráfego desconhecido para `r4_edge` porque ele é o gateway da internet. O próprio `r4_edge` já tem a internet na sua interface (203.0.113.0/24) — não existe roteador acima dele que conheça mais destinos. Uma rota padrão no `r4_edge` não teria para onde apontar de forma útil.

Por que o r4_edge precisa das rotas internas? O `r4_edge` realiza NAT: quando um host interno acessa a internet, o IP de origem é reescrito para 203.0.113.1. Quando a resposta chega, o `r4_edge` precisa desfazer a tradução e entregar o pacote de volta ao host original. Para isso, precisa saber como alcançar as sub-redes internas:

None

192.168.100.64/26 via 192.168.100.2 ← pacotes para Polo 1 vão por r2_br1

192.168.100.128/26 via 192.168.100.3 ← pacotes para Polo 2 vão por r3_br2

192.168.100.192/26 via 192.168.100.1 ← pacotes para Gerência vão por r1_hq

Sem essas rotas, as respostas da internet chegariam ao `r4_edge` com um IP de destino privado e seriam descartadas, pois ele não saberia para onde encaminhá-las.

Implementação

As configurações foram implementadas no script `scripts/03_routing/run.sh` usando dois comandos:

Shell

```
# Rota padrão para hosts – todo tráfego desconhecido vai para o roteador local
```

```
docker exec pc1_br1 ip route add default via 192.168.100.65
```

```
# Rota estática para roteadores – tráfego para Polo 2 vai para r3_br2
```

```
docker exec r2_br1 ip route add 192.168.100.128/26 via 192.168.100.3
```

O `ip route add default via <gateway>` instrui o kernel a encaminhar qualquer pacote sem rota específica para o endereço indicado. A partir daí, o roteador local assume a responsabilidade de encontrar o destino.

O `ip route add <sub-rede> via <next-hop>` cria uma rota específica: pacotes destinados àquela sub-rede são encaminhados ao next-hop indicado. O next-hop precisa ser um endereço diretamente alcançável — no caso dos roteadores, todos os next-hops são IPs do backbone `net_servico`, que é uma sub-rede compartilhada por todos eles.

Verificação

A verificação foi implementada no script `scripts/03_routing/test.sh` em três níveis. O **Nível 1** exibe a tabela de roteamento completa de cada dispositivo com `ip route`, confirmando que a rota padrão e as rotas estáticas foram criadas. O **Nível 2** testa a conectividade entre sub-redes com `ping`. Neste ponto o firewall ainda não foi aplicado, portanto todos os pings devem passar — inclusive aqueles que serão bloqueados no passo 5 (Polo 1 ↔ Polo 2, Polos → Gerência). O **Nível 3** executa um `traceroute` de `pc1_br1` até `srv_dns` para confirmar o caminho exato dos pacotes:

None

```
traceroute to 192.168.100.10, 30 hops max
1 192.168.100.65 ← r2_br1 (gateway de Polo 1)
2 192.168.100.10 ← srv_dns (destino)
```

As tabelas de roteamento após a configuração ficaram:

r1_hq

None

```
default via 192.168.100.4 dev eth1 ← internet via r4_edge
192.168.100.0/26 dev eth1 proto kernel src .1 ← net_servico
(conectada)
192.168.100.64/26 via 192.168.100.2 dev eth1 ← Polo 1 via r2_br1
192.168.100.128/26 via 192.168.100.3 dev eth1 ← Polo 2 via r3_br2
```

```
192.168.100.192/26 dev eth0 proto kernel src .193 ← net_gerencia  
(conectada)
```

r2_br1

None

```
default via 192.168.100.4 dev eth0 ← internet via r4_edge  
192.168.100.0/26 dev eth0 proto kernel src .2 ← net_servico  
(conectada)  
192.168.100.64/26 dev eth1 proto kernel src .65 ← net_polo1  
(conectada)  
192.168.100.128/26 via 192.168.100.3 dev eth0 ← Polo 2 via r3_br2  
192.168.100.192/26 via 192.168.100.1 dev eth0 ← Gerência via  
r1_hq
```

r3_br2

None

```
default via 192.168.100.4 dev eth1 ← internet via r4_edge  
192.168.100.0/26 dev eth1 proto kernel src .3 ← net_servico  
(conectada)  
192.168.100.64/26 via 192.168.100.2 dev eth1 ← Polo 1 via r2_br1  
192.168.100.128/26 dev eth0 proto kernel src .129 ← net_polo2  
(conectada)  
192.168.100.192/26 via 192.168.100.1 dev eth1 ← Gerência via  
r1_hq
```

r4_edge

None

```
192.168.100.0/26 dev eth0 proto kernel src .4 ← net_servico  
(conectada)  
192.168.100.64/26 via 192.168.100.2 dev eth0 ← Polo 1 via r2_br1  
192.168.100.128/26 via 192.168.100.3 dev eth0 ← Polo 2 via r3_br2  
192.168.100.192/26 via 192.168.100.1 dev eth0 ← Gerência via  
r1_hq  
203.0.113.0/24 dev eth1 proto kernel src .1 ← net_internet  
(conectada)
```

Hosts (exemplo representativo por sub-rede)

None

```
pc1_br1 → default via 192.168.100.65 | 192.168.100.64/26 dev eth0  
src .66
```

```
pc1_br2 → default via 192.168.100.129 | 192.168.100.128/26 dev
eth0 src .130
admin_pc → default via 192.168.100.193 | 192.168.100.192/26 dev
eth0 src .144
srv_dns → default via 192.168.100.4 | 192.168.100.0/26 dev eth0
src .10
```

5 NAT (Network Address Translation)

A implementação do NAT (Network Address Translation) foi essencial, pois os hosts internos operam com endereços IP privados (192.168.100.x), não roteáveis na internet pública. Sem a tradução de endereços, as respostas vindas de servidores externos seriam descartadas por roteadores na internet por não possuírem uma rota de retorno válida. A configuração do NAT no `r4_edge` resolve essa assimetria, permitindo que o tráfego de saída seja traduzido para um IP público válido.

Configuração

Uma única regra `iptables` no `r4_edge` resolve o NAT para toda a rede interna:

```
Shell
docker exec r4_edge iptables -t nat -A POSTROUTING -o
<interface_internet> -j MASQUERADE
```

Parâmetro	Significado
<code>-t nat</code>	Opera na tabela NAT do iptables
<code>-A POSTROUTING</code>	Aplicado imediatamente antes do pacote sair da máquina
<code>-o <interface></code>	Apenas para pacotes saindo pela interface de internet
<code>-j MASQUERADE</code>	Reescreve o IP de origem com o IP atual da interface de saída

Optou-se pelo uso do **MASQUERADE**, uma forma dinâmica de SNAT (Source NAT). A principal vantagem é que ele ajusta o endereço de origem automaticamente para o IP da interface de saída (203.0.113.1), eliminando a necessidade de fixar o IP público na regra. O `r4_edge` gerencia uma tabela de estado de conexões para reverter a tradução quando os pacotes de resposta retornam da internet.

Nota: a interface de internet é detectada em tempo de execução pelo mesmo mecanismo do passo 1 — a interface pode ser `eth0` ou `eth1` dependendo do que o Docker atribuiu nessa execução.

Como o NAT funciona

O fluxo completo de um pacote de `pc1_br1` até `srv_public_web`:

None

1. `pc1_br1` envia: `src=192.168.100.66 dst=203.0.113.10`
2. `r2_br1` encaminha: `src=192.168.100.66 dst=203.0.113.10` (roteamento normal)
3. `r4_edge` reescreve: `src=203.0.113.1 dst=203.0.113.10` → sai para internet
4. `srv_public_web` responde: `src=203.0.113.10 dst=203.0.113.1`
5. `r4_edge` restaura: `src=203.0.113.10 dst=192.168.100.66` → encaminha para `r2_br1`
6. `r2_br1` entrega: pacote chega em `pc1_br1`

A tradução de endereço é completamente transparente para o host interno. Para o servidor de destino (`srv_public_web`), todos os clientes corporativos aparecem com o mesmo IP público: `203.0.113.1`, o endereço do `r4_edge`.

Por que apenas o `r4_edge` faz NAT?

O NAT precisa estar no ponto de fronteira entre a rede privada e a internet. O `r4_edge` é o único roteador com uma interface em cada lado:

None

```
eth? → net_servico (192.168.100.0/26) – rede privada
eth? → net_internet (203.0.113.0/24) – internet
```

Os demais roteadores (`r1_hq`, `r2_br1`, `r3_br2`) conectam apenas sub-redes privadas entre si e nunca tocam a internet, portanto não precisam de NAT.

Verificação

A verificação foi implementada no script `scripts/04_nat/test.sh` em três níveis.

O **Nível 1** confirma que a regra `MASQUERADE` existe na tabela NAT do `r4_edge`:

Shell

```
docker exec r4_edge iptables -t nat -L POSTROUTING | grep MASQUERADE
# Output: MASQUERADE all -- anywhere anywhere
```

O **Nível 2** testa a conectividade de todos os hosts internos com `srv_public_web` (`203.0.113.10`) via `ping`. Um timeout indica que o NAT não está traduzindo corretamente.

O **Nível 3** executa um `traceroute` de `pc1_br1` até `srv_public_web` para confirmar o caminho completo:

None

```
traceroute to 203.0.113.10, 30 hops max
1 192.168.100.65 ← r2_br1 (gateway de Polo 1)
2 192.168.100.4 ← r4_edge (backbone)
3 203.0.113.10 ← srv_public_web (destino)
```

O salto 2 mostra o `r4_edge` pelo seu IP do backbone (192.168.100.4). A tradução NAT acontece nesse ponto — após o salto 2, o pacote sai com IP de origem 203.0.113.1, mas o `traceroute` não mostra isso porque o ICMP TTL exceeded é gerado antes da reescrita.

6 Regras de Firewall (iptables)

Com a conectividade e o acesso à internet plenamente funcionais, a rede se encontra em um estado 'aberto' por padrão. Esta etapa final é dedicada à aplicação da política de segurança corporativa por meio de regras de *firewall stateful* (`iptables`). O foco é estabelecer o isolamento obrigatório entre as filiais, proteger os servidores sensíveis (Banco de Dados e DNS) e, crucialmente, impedir qualquer tentativa de conexão externa iniciada pela internet.

Configurações

O ponto de aplicação das nossas políticas é a chain **FORWARD** do Netfilter. Esta *chain* é dedicada exclusivamente ao processamento de pacotes que atravessam o roteador, ou seja, que entram por uma interface e se destinam a sair por outra (tráfego que *passa*). A lógica de processamento do `iptables` é sequencial (*top-down*), e a primeira regra correspondente é aplicada. Por isso, a regra de estado (`-m state --state ESTABLISHED,RELATED -j ACCEPT`) é posicionada como a **primeira** em todos os roteadores. Isso garante que as respostas a sessões legítimas já estabelecidas sejam aceitas imediatamente, evitando que sejam descartadas por regras de bloqueio genéricas que vêm na sequência.

Abaixo estão as tabelas de bloqueio (políticas `DROP`) configuradas:

Roteador	Alvo/Bloqueio	Motivação
r2_br1	Polo 1 ↔ Polo 2 e Gerência	Isolamento de departamentos
r3_br2	Polo 2 ↔ Polo 1 e Gerência	Isolamento de departamentos
r1_hq	Polos → Gerência	Restrição de acesso administrativo

Roteador	Alvo/Bloqueio	Motivação
r4_edge	Internet → Rede Interna	Bloqueio de novas conexões externas

Proteção de borda. No `r4_edge` foi implementada uma regra específica para descartar pacotes vindos da interface de internet que tentem iniciar novas conexões (estado `NEW`) com destino a qualquer sub-rede privada, garantindo a invisibilidade dos hosts internos para atacantes externos.

Implementação

A regra de bloqueio de borda no `r4_edge` requer a identificação dinâmica da interface de internet (que pode ser `eth0` ou `eth1`). O script reutiliza a lógica de detecção de interface introduzida no Passo 1, buscando a interface associada ao bloco `203.0.113.x`. Isso garante que a política de segurança (`-i $R4_INTERNET -m state --state NEW -j DROP`) seja aplicada corretamente, bloqueando o início de qualquer conexão vinda da internet.

Shell

```
# r1_hq: Impede que Polos acessem a Gerência
docker exec r1_hq iptables -A FORWARD -s 192.168.100.64/26 -d 192.168.100.192/26 -j DROP
docker exec r1_hq iptables -A FORWARD -s 192.168.100.128/26 -d 192.168.100.192/26 -j DROP
# r2_br1: Isolamento do Polo 1
docker exec r2_br1 iptables -A FORWARD -s 192.168.100.64/26 -d 192.168.100.128/26 -j DROP
docker exec r2_br1 iptables -A FORWARD -s 192.168.100.64/26 -d 192.168.100.192/26 -j DROP
# r4_edge: Proteção contra tráfego externo NEW
docker exec r4_edge iptables -A FORWARD -i eth1 -m state --state NEW -j DROP
```

Verificação

A validação final foi realizada através de testes de conectividade divididos em três grupos principais, confirmando a eficácia das regras. O `iptables -L FORWARD -v` pode ser usado para inspecionar as regras ativas.

- **Grupo 1: Tráfego Permitido.** Confirma que Polos e Gerência acessam os servidores e a internet.
- **Grupo 2: Isolamento Interno.** Garante que o `ping` falha entre Polos e entre Polos e Gerência.

- **Grupo 3: Segurança de Borda.** Confirma que conexões originadas na internet para a rede interna são descartadas.

7 Conclusão

A conclusão do projeto demonstra a implementação bem-sucedida de uma arquitetura de rede corporativa completa, que atende a todos os requisitos de conectividade e segurança propostos. A progressão das cinco fases garantiu a transição de uma infraestrutura base (endereçamento IP) para uma rede totalmente funcional e segura.

Iniciamos com a **Atribuição de IPs** (Seções 2 e 3), onde o desafio do mapeamento não-determinístico de interfaces do Docker foi superado através da detecção de interfaces em tempo de execução, assegurando que todos os endereços estáticos fossem aplicados às sub-redes corretas.

Em seguida, o **Roteamento Estático** (Seção 4) estabeleceu a inteligência da Camada 3. Ao configurar o *default gateway* em todos os hosts e rotas estáticas nos roteadores (utilizando IPs do *backbone* como *next-hops*), garantimos a comunicação total entre todas as sub-redes internas. A otimização do caminho de saída para os servidores (apontando diretamente para o `r4_edge`) demonstrou uma escolha estratégica para a eficiência do tráfego.

A conectividade externa foi resolvida com a implementação do **NAT** (Seção 5) no roteador de borda. A regra `MASQUERADE` do iptables permitiu que todos os hosts internos, utilizando IPs privados, acessassem a internet com transparência, superando o desafio de roteamento de pacotes de retorno.

Finalmente, a **Segurança da Rede** (Seção 6) foi garantida pela aplicação de regras *stateful* na *chain FORWARD* do iptables em todos os roteadores. As políticas implementadas asseguraram o isolamento bidirecional entre as filiais (Polo 1 e Polo 2), protegeram os servidores críticos (DNS e DB) do acesso por hosts administrativos e de filiais, e, fundamentalmente, bloquearam qualquer tentativa de conexão *nova* iniciada por entidades externas (internet), cumprindo integralmente os requisitos de isolamento e proteção de dados sensíveis.\

Anexos

01_routers/run.sh

```
#!/bin/bash
# =====
# 01_routers/run.sh - Assign IPs to routers
#
# Docker does not guarantee which interface (eth0 or eth1) is
# connected to which network - the assignment varies between runs
# and the priority field in docker-compose.yml is not reliable.
#
# Strategy: Docker always assigns a temporary 172.x.x.x IP to each
# container interface when it starts. We use that IP to identify
# which interface belongs to which network, then assign our static
# IPs to the correct interfaces.
#
# Logical assignments (IPs never change; only the interface name may):
#   r1_hq   : net_gerencia=192.168.100.193/26   net_servico=192.168.100.1/26
#   r2_br1  : net_polol1=192.168.100.65/26      net_servico=192.168.100.2/26
#   r3_br2  : net_polol2=192.168.100.129/26     net_servico=192.168.100.3/26
#   r4_edge : net_internet=203.0.113.1/24       net_servico=192.168.100.4/26
# =====

run_cmd() {
    local reason="$1"
    shift
    printf "    Reason  : %s\n" "$reason"
    printf "    Command : %s\n" "$*"
    local output
    output=$(("$@" 2>&1)
    local exit_code=$?
    if [ -n "$output" ]; then
        echo "$output" | sed 's/^/    Output : /'
    else
        printf "    Output : (no output)\n"
    fi
    if [ "$exit_code" -eq 0 ]; then
        printf "    Status : done (exit 0)\n\n"
    else
        printf "    Status : FAILED (exit %d)\n\n" "$exit_code"
    fi
    return $exit_code
}

# find_iface CONTAINER NETWORK
#
# Returns the interface name (eth0 or eth1) that is connected to NETWORK.
#
# How it works:
#   1. Ask Docker which 172.x.x.x IP it assigned to CONTAINER on NETWORK
#   2. Look for that IP inside the container to identify the interface
find_iface() {
    local container="$1"
    local network="student-environment_$2"
```

```

local docker_ip
docker_ip=$(docker network inspect "$network" \
    | grep -A4 "\"Name\": \"\${container}\"" \
    | grep "IPv4Address" \
    | grep -oE '172\.[0-9]+\.[0-9]+\.[0-9]+')

docker exec "$container" ip addr show \
    | awk -v ip="$docker_ip" '
        /^[0-9]+:/ { split($2, a, "@"); iface = a[1]; gsub(/:$/, "", iface) }
        /inet /    { if ($2 ~ ip) print iface }
    '
}

echo ""
echo "=====
echo "  STEP 1 - Router IP Assignment"
echo "=====
echo "  Routers are the only devices with two interfaces."
echo "  Each interface connects to a different subnet."
echo "  Interfaces are detected at runtime because Docker does"
echo "  not guarantee which interface maps to which network."
echo "=====

# -- r1_hq -----
echo ""
echo "  [r1_hq] Headquarters Router -----"
echo "  | Role: connects Management to the backbone |"
echo "  | ip_forward=1 allows it to route between both |"
echo "  |-----|"
echo ""

R1_GERENCIA=$(find_iface r1_hq net_gerencia)
R1_SERVICO=$(find_iface r1_hq net_servico)
printf "    Detected: net_gerencia=%s net_servico=%s\n\n" "$R1_GERENCIA" "$R1_SERVICO"

echo "  $R1_GERENCIA -> net_gerencia | 192.168.100.192/26 | Range: .193-.254"
echo ""
run_cmd "Remove Docker's auto-assigned IP from $R1_GERENCIA" \
    docker exec r1_hq ip addr flush dev "$R1_GERENCIA"
run_cmd "Assign gateway IP for net_gerencia - admin_pc will use this as default gateway" \
    docker exec r1_hq ip addr add 192.168.100.193/26 dev "$R1_GERENCIA"

echo "  $R1_SERVICO -> net_servico | 192.168.100.0/26 | Range: .1-.62"
echo ""
run_cmd "Remove Docker's auto-assigned IP from $R1_SERVICO" \
    docker exec r1_hq ip addr flush dev "$R1_SERVICO"
run_cmd "Assign backbone identity of r1_hq - other routers use this IP as next-hop" \
    docker exec r1_hq ip addr add 192.168.100.1/26 dev "$R1_SERVICO"

# -- r2_br1 -----
echo ""
echo "  [r2_br1] Branch 1 Router -----"
echo "  | Role: connects Polo 1 clients to the backbone |"
echo "  | ip_forward=1 allows it to route between both |"
echo "  |-----|"
echo ""

```

```

R2_POLO1=$(find_iface r2_br1 net_polo1)
R2_SERVICO=$(find_iface r2_br1 net_servico)
printf "    Detected: net_polo1=%s net_servico=%s\n\n" "$R2_POLO1" "$R2_SERVICO"

echo "    $R2_POLO1 → net_polo1 | 192.168.100.64/26 | Range: .65-.126"
echo ""
run_cmd "Remove Docker's auto-assigned IP from $R2_POLO1" \
    docker exec r2_br1 ip addr flush dev "$R2_POLO1"
run_cmd "Assign gateway IP for net_polo1 – pc1/2/3_br1 will use this as default gateway" \
    docker exec r2_br1 ip addr add 192.168.100.65/26 dev "$R2_POLO1"

echo "    $R2_SERVICO → net_servico | 192.168.100.0/26 | Range: .1-.62"
echo ""
run_cmd "Remove Docker's auto-assigned IP from $R2_SERVICO" \
    docker exec r2_br1 ip addr flush dev "$R2_SERVICO"
run_cmd "Assign backbone identity of r2_br1 – other routers use this IP as next-hop" \
    docker exec r2_br1 ip addr add 192.168.100.2/26 dev "$R2_SERVICO"

# --- r3_br2 -----
echo ""
echo "    ┌ [r3_br2] Branch 2 Router ────────────────────────────────────┐"
echo "    │ Role: connects Polo 2 clients to the backbone                 │"
echo "    │ ip_forward=1 allows it to route between both                  │"
echo "    └──────────────────────────────────────────────────────────────────┘"
echo ""

R3_POLO2=$(find_iface r3_br2 net_polo2)
R3_SERVICO=$(find_iface r3_br2 net_servico)
printf "    Detected: net_polo2=%s net_servico=%s\n\n" "$R3_POLO2" "$R3_SERVICO"

echo "    $R3_POLO2 → net_polo2 | 192.168.100.128/26 | Range: .129-.190"
echo ""
run_cmd "Remove Docker's auto-assigned IP from $R3_POLO2" \
    docker exec r3_br2 ip addr flush dev "$R3_POLO2"
run_cmd "Assign gateway IP for net_polo2 – pc1/2/3_br2 will use this as default gateway" \
    docker exec r3_br2 ip addr add 192.168.100.129/26 dev "$R3_POLO2"

echo "    $R3_SERVICO → net_servico | 192.168.100.0/26 | Range: .1-.62"
echo ""
run_cmd "Remove Docker's auto-assigned IP from $R3_SERVICO" \
    docker exec r3_br2 ip addr flush dev "$R3_SERVICO"
run_cmd "Assign backbone identity of r3_br2 – other routers use this IP as next-hop" \
    docker exec r3_br2 ip addr add 192.168.100.3/26 dev "$R3_SERVICO"

# --- r4_edge -----
echo ""
echo "    ┌ [r4_edge] Edge Router (Internet Gateway) ────────────────────┐"
echo "    │ Role: border between internal network and internet             │"
echo "    │ Will perform NAT so internal hosts reach internet              │"
echo "    │ ip_forward=1 allows it to route between both                  │"
echo "    └──────────────────────────────────────────────────────────────────┘"
echo ""

R4_INTERNET=$(find_iface r4_edge net_internet)
R4_SERVICO=$(find_iface r4_edge net_servico)
printf "    Detected: net_internet=%s net_servico=%s\n\n" "$R4_INTERNET" "$R4_SERVICO"

```

```

echo "  $R4_INTERNET → net_internet | 203.0.113.0/24 | Range: .1-.254"
echo ""
run_cmd "Remove Docker's auto-assigned IP from $R4_INTERNET" \
  docker exec r4_edge ip addr flush dev "$R4_INTERNET"
run_cmd "Assign public-facing IP – internal hosts appear as this after NAT" \
  docker exec r4_edge ip addr add 203.0.113.1/24 dev "$R4_INTERNET"

echo "  $R4_SERVICO → net_servico | 192.168.100.0/26 | Range: .1-.62"
echo ""
run_cmd "Remove Docker's auto-assigned IP from $R4_SERVICO" \
  docker exec r4_edge ip addr flush dev "$R4_SERVICO"
run_cmd "Assign backbone identity of r4_edge – internal routers use this as next-hop for
internet" \
  docker exec r4_edge ip addr add 192.168.100.4/26 dev "$R4_SERVICO"

echo "====="
echo "  Router IPs assigned."
echo "  Run: bash scripts/01_routers/test.sh   to verify"
echo "====="

```

01_routers/test.sh

```

#!/bin/bash
# =====
# 01_routers/test.sh – Verify router IP assignment
# =====

PASS=0
FAIL=0

# check_ip CONTAINER EXPECTED_IP DESCRIPTION
#
# Searches for EXPECTED_IP on any interface of CONTAINER.
# Reports which interface the IP was found on (informational).
# Does not require knowing the interface name in advance because
# the 172.x.x.x Docker IPs are gone after run.sh flushes them.
check_ip() {
  local container=$1
  local expected_ip=$2
  local description=$3

  printf "    Reason  : Confirm %s has IP %s assigned\n" "$container" "$expected_ip"
  printf "    Command : docker exec %s ip addr show | grep %s\n" "$container"
"$expected_ip"

  local output iface
  output=$(docker exec "$container" ip addr show 2>/dev/null | grep "$expected_ip")
  iface=$(docker exec "$container" ip addr show 2>/dev/null | awk -v ip="$expected_ip" '
    /^[0-9]+:/ { split($2, a, "@"); iface = a[1]; gsub(/:$/, "", iface) }
    /inet /    { if ($2 ~ ip) print iface }
  ')

  if [ -n "$output" ]; then

```

```

        echo "$output" | sed 's/^/    Output : /'
        printf "        Status : [PASS] %s has %s on %s - %s\n\n" "$container" "$expected_ip"
"$iface" "$description"
        PASS=$((PASS + 1))
    else
        printf "        Output : (not found)\n"
        printf "        Status : [FAIL] %s is MISSING %s - %s\n\n" "$container" "$expected_ip"
"$description"
        FAIL=$((FAIL + 1))
    fi
}

check_ping() {
    local from=$1
    local to_ip=$2
    local to_name=$3

    printf "        Reason : Routers on the same subnet must reach each other without any
routing config\n"
    printf "        Command : docker exec %s ping -c1 -W1 %s\n" "$from" "$to_ip"
    local output
    output=$(docker exec "$from" ping -c1 -W1 "$to_ip" 2>&1)
    local exit_code=$?
    local summary
    summary=$(echo "$output" | grep -E "packets transmitted" | head -1)
    printf "        Output : %s\n" "$summary"
    if [ "$exit_code" -eq 0 ]; then
        printf "        Status : [PASS] %s → %s (%s) reachable\n\n" "$from" "$to_ip" "$to_name"
        PASS=$((PASS + 1))
    else
        printf "        Status : [FAIL] %s → %s (%s) UNREACHABLE\n\n" "$from" "$to_ip"
"$to_name"
        FAIL=$((FAIL + 1))
    fi
}

show_routes() {
    local device=$1
    printf "        Reason : Confirm kernel auto-created connected routes after IP
assignment\n"
    printf "        Command : docker exec %s ip route\n" "$device"
    local output
    output=$(docker exec "$device" ip route 2>&1)
    echo "$output" | sed 's/^/    Output : /'
    printf "        Status : displayed\n\n"
}

echo ""
echo "=====
echo "  TEST 1 – Router IP Assignment Verification
echo "=====

# -- Level 1: IPs are present on the container -----
echo ""
echo " ┌ Level 1 – IP Assignment -----┐"
echo " │ Checks that each static IP is assigned. │"
echo " │ Also reports which interface it landed on. │"

```

```

echo " [ ] "

echo ""
echo " [r1_hq]"
check_ip r1_hq 192.168.100.193 "net_gerencia - gateway for admin_pc"
check_ip r1_hq 192.168.100.1 "net_servico - backbone identity"

echo " [r2_br1]"
check_ip r2_br1 192.168.100.65 "net_polo1 - gateway for pc1/2/3_br1"
check_ip r2_br1 192.168.100.2 "net_servico - backbone identity"

echo " [r3_br2]"
check_ip r3_br2 192.168.100.129 "net_polo2 - gateway for pc1/2/3_br2"
check_ip r3_br2 192.168.100.3 "net_servico - backbone identity"

echo " [r4_edge]"
check_ip r4_edge 203.0.113.1 "net_internet - public-facing, will do NAT"
check_ip r4_edge 192.168.100.4 "net_servico - backbone identity"

# -- Level 2: Routers can reach each other on net_servico --
echo ""
echo " [ Level 2 - Reachability on net_servico ] "
echo " | All routers share 192.168.100.0/26 on net_servico. | "
echo " | No routing config needed - same subnet = direct. | "
echo " | A failure here means an IP is on the wrong bridge. | "
echo " [ ] "
echo ""

check_ping r1_hq 192.168.100.2 "r2_br1"
check_ping r1_hq 192.168.100.3 "r3_br2"
check_ping r1_hq 192.168.100.4 "r4_edge"
check_ping r2_br1 192.168.100.1 "r1_hq"
check_ping r2_br1 192.168.100.3 "r3_br2"
check_ping r2_br1 192.168.100.4 "r4_edge"
check_ping r3_br2 192.168.100.1 "r1_hq"
check_ping r3_br2 192.168.100.2 "r2_br1"
check_ping r3_br2 192.168.100.4 "r4_edge"
check_ping r4_edge 192.168.100.1 "r1_hq"
check_ping r4_edge 192.168.100.2 "r2_br1"
check_ping r4_edge 192.168.100.3 "r3_br2"

# -- Level 3: Kernel routing tables --
echo ""
echo " [ Level 3 - Kernel Routing Tables ] "
echo " | Assigning an IP auto-creates a connected route in | "
echo " | the kernel. No static config needed. | "
echo " [ ] "
echo ""

for router in r1_hq r2_br1 r3_br2 r4_edge; do
    echo " [$router]"
    show_routes "$router"
done

echo "====="
if [ "$FAIL" -eq 0 ]; then
    echo " ALL $PASS checks PASSED"

```



```

    echo "    Routers are correctly configured."
    echo "    Next: bash scripts/02_hosts/run.sh"
else
    echo "    $PASS passed | $FAIL FAILED"
    echo "    Fix the issues above before continuing."
fi
echo "=====
```

02_hosts/run.sh

```

#!/bin/bash
# =====
# 02_hosts/run.sh - Assign IPs to all end-host devices
#
# End hosts have only ONE interface (eth0), so they get
# a single IP within their subnet.
# =====

run_cmd() {
    local reason="$1"
    shift
    printf "    Reason  : %s\n" "$reason"
    printf "    Command : %s\n" "$*"
    local output
    output=$(("$@" 2>&1)
    local exit_code=$?
    if [ -n "$output" ]; then
        echo "$output" | sed 's/^/    Output  : /'
    else
        printf "    Output  : (no output)\n"
    fi
    if [ "$exit_code" -eq 0 ]; then
        printf "    Status  : done (exit 0)\n\n"
    else
        printf "    Status  : FAILED (exit %d)\n\n" "$exit_code"
    fi
    return $exit_code
}

echo ""
echo "=====
echo "    STEP 2 - Host IP Assignment"
echo "=====
echo "    End hosts have a single interface (eth0)."
echo "    Unlike routers, they do not forward traffic."
echo "    Each host gets one IP within its subnet."
echo "=====

# -- net_servico - Corporate servers -----
echo ""
echo "    ┌ [net_servico] Corporate Servers ────────────┐"
echo "    │ Subnet   : 192.168.100.0/26                    │"
echo "    │ Range    : 192.168.100.1 - 192.168.100.62      │"
echo "    │ Gateway  : 192.168.100.1 (r1_hq eth1)          │"
```

```

echo " _____"
echo ""

run_cmd "Remove Docker's auto-assigned IP from srv_dns" \
    docker exec srv_dns ip addr flush dev eth0
run_cmd "Assign static IP to srv_dns - access will be restricted by firewall rules in step 5" \
    docker exec srv_dns ip addr add 192.168.100.10/26 dev eth0

run_cmd "Remove Docker's auto-assigned IP from srv_web" \
    docker exec srv_web ip addr flush dev eth0
run_cmd "Assign static IP to srv_web - Polo 2 is allowed to reach this server on port 80" \
    docker exec srv_web ip addr add 192.168.100.11/26 dev eth0

run_cmd "Remove Docker's auto-assigned IP from srv_db" \
    docker exec srv_db ip addr flush dev eth0
run_cmd "Assign static IP to srv_db - access will be restricted by firewall rules in step 5" \
    docker exec srv_db ip addr add 192.168.100.12/26 dev eth0

# --- net_polo1 - Branch 1 clients -----
echo ""
echo " ┌ [net_polo1] Branch 1 Client PCs ────────────┐"
echo " │ Subnet   : 192.168.100.64/26                    │"
echo " │ Range    : 192.168.100.65 - 192.168.100.126      │"
echo " │ Gateway  : 192.168.100.65 (r2_br1 eth0)          │"
echo " └──────────────────────────────────────────────────┘"
echo ""

run_cmd "Remove Docker's auto-assigned IP from pc1_br1" \
    docker exec pc1_br1 ip addr flush dev eth0
run_cmd "Assign static IP to pc1_br1 within net_polo1 subnet" \
    docker exec pc1_br1 ip addr add 192.168.100.66/26 dev eth0

run_cmd "Remove Docker's auto-assigned IP from pc2_br1" \
    docker exec pc2_br1 ip addr flush dev eth0
run_cmd "Assign static IP to pc2_br1 within net_polo1 subnet" \
    docker exec pc2_br1 ip addr add 192.168.100.67/26 dev eth0

run_cmd "Remove Docker's auto-assigned IP from pc3_br1" \
    docker exec pc3_br1 ip addr flush dev eth0
run_cmd "Assign static IP to pc3_br1 within net_polo1 subnet" \
    docker exec pc3_br1 ip addr add 192.168.100.68/26 dev eth0

# --- net_polo2 - Branch 2 clients -----
echo ""
echo " ┌ [net_polo2] Branch 2 Client PCs ────────────┐"
echo " │ Subnet   : 192.168.100.128/26                    │"
echo " │ Range    : 192.168.100.129 - 192.168.100.190      │"
echo " │ Gateway  : 192.168.100.129 (r3_br2 eth0)          │"
echo " └──────────────────────────────────────────────────┘"
echo ""

run_cmd "Remove Docker's auto-assigned IP from pc1_br2" \
    docker exec pc1_br2 ip addr flush dev eth0
run_cmd "Assign static IP to pc1_br2 within net_polo2 subnet" \
    docker exec pc1_br2 ip addr add 192.168.100.130/26 dev eth0

```

```

run_cmd "Remove Docker's auto-assigned IP from pc2_br2" \
    docker exec pc2_br2 ip addr flush dev eth0
run_cmd "Assign static IP to pc2_br2 within net_polo2 subnet" \
    docker exec pc2_br2 ip addr add 192.168.100.131/26 dev eth0

run_cmd "Remove Docker's auto-assigned IP from pc3_br2" \
    docker exec pc3_br2 ip addr flush dev eth0
run_cmd "Assign static IP to pc3_br2 within net_polo2 subnet" \
    docker exec pc3_br2 ip addr add 192.168.100.132/26 dev eth0

# -- net_gerencia - Management -----
echo ""
echo "  ┌ [net_gerencia] Management Network ────────────┐"
echo "  │ Subnet   : 192.168.100.192/26                    │"
echo "  │ Range    : 192.168.100.193 - 192.168.100.254      │"
echo "  │ Gateway  : 192.168.100.193 (r1_hq eth0)           │"
echo "  └──────────────────────────────────────────────────┘"
echo ""

run_cmd "Remove Docker's auto-assigned IP from admin_pc" \
    docker exec admin_pc ip addr flush dev eth0
run_cmd "Assign static IP to admin_pc - firewall will isolate this from all Polo subnets" \
    docker exec admin_pc ip addr add 192.168.100.194/26 dev eth0

# -- net_internet - Public internet devices -----
echo ""
echo "  ┌ [net_internet] Simulated Public Internet ───────────┐"
echo "  │ Subnet    : 203.0.113.0/24                          │"
echo "  │ Gateway   : 203.0.113.1 (r4_edge eth0)               │"
echo "  └──────────────────────────────────────────────────┘"
echo ""

run_cmd "Remove Docker's auto-assigned IP from srv_public_web" \
    docker exec srv_public_web ip addr flush dev eth0
run_cmd "Assign public IP to srv_public_web - internal hosts will reach it via NAT through r4_edge" \
    docker exec srv_public_web ip addr add 203.0.113.10/24 dev eth0

run_cmd "Remove Docker's auto-assigned IP from ext_client" \
    docker exec ext_client ip addr flush dev eth0
run_cmd "Assign public IP to ext_client - simulates an external attacker blocked by stateful firewall" \
    docker exec ext_client ip addr add 203.0.113.20/24 dev eth0

echo "=====
echo "  Host IPs assigned."
echo "  Run: bash scripts/02_hosts/test.sh   to verify"
echo "=====

```

02_hosts/test.sh

```

#!/bin/bash
# =====

```

```
# 02_hosts/test.sh - Verify all host IPs were assigned
# =====

PASS=0
FAIL=0

check() {
    local container=$1
    local expected_ip=$2
    local description=$3

    printf "    Reason  : Confirm %s has IP %s assigned\n" "$container" "$expected_ip"
    printf "    Command : docker exec %s ip addr show | grep %s\n" "$container"
"$expected_ip"
    local output
    output=$(docker exec "$container" ip addr show 2>/dev/null | grep "$expected_ip")
    if [ -n "$output" ]; then
        echo "$output" | sed 's/^/    Output  : /'
        printf "    Status  : [PASS] %s has %s - %s\n\n" "$container" "$expected_ip"
"$description"
        PASS=$((PASS + 1))
    else
        printf "    Output  : (not found)\n"
        printf "    Status  : [FAIL] %s is MISSING %s - %s\n\n" "$container" "$expected_ip"
"$description"
        FAIL=$((FAIL + 1))
    fi
}

show_routes() {
    local device=$1
    printf "    Reason  : Confirm kernel auto-created a connected route - no default route yet\n"
    printf "    Command : docker exec %s ip route\n" "$device"
    local output
    output=$(docker exec "$device" ip route 2>&1)
    echo "$output" | sed 's/^/    Output  : /'
    printf "    Status  : displayed\n\n"
}

echo ""
echo "=====
echo "  TEST 2 - Host IP Assignment Verification
echo "=====

echo ""
echo "  ┌ Corporate Servers - net_servico (192.168.100.0/26) ┐"
echo "  │ Gateway: 192.168.100.1 (r1_hq eth1) │"
echo "  └───────────────────────────────────────────────────┘"

check srv_dns 192.168.100.10 "DNS server"
check srv_web 192.168.100.11 "Web server"
check srv_db 192.168.100.12 "Database - access restricted by firewall"

echo ""
echo "  ┌ Branch 1 Clients - net_polo1 (192.168.100.64/26) ┐"
echo "  │ Gateway: 192.168.100.65 (r2_br1 eth0) │"
echo "  └───────────────────────────────────────────────────┘"
```

```

check pc1_br1 192.168.100.66 "client PC 1"
check pc2_br1 192.168.100.67 "client PC 2"
check pc3_br1 192.168.100.68 "client PC 3"

echo ""
echo " ┌ Branch 2 Clients - net_polo2 (192.168.100.128/26) ─┐"
echo " │ Gateway: 192.168.100.129 (r3_br2 eth0) │"
echo " └──────────────────────────────────────────────────┘"

check pc1_br2 192.168.100.130 "client PC 1"
check pc2_br2 192.168.100.131 "client PC 2"
check pc3_br2 192.168.100.132 "client PC 3"

echo ""
echo " ┌ Management - net_gerencia (192.168.100.192/26) ───┐"
echo " │ Gateway: 192.168.100.193 (r1_hq eth0) │"
echo " └──────────────────────────────────────────────────┘"

check admin_pc 192.168.100.194 "admin workstation"

echo ""
echo " ┌ Internet Devices - net_internet (203.0.113.0/24) ───┐"
echo " │ Gateway: 203.0.113.1 (r4_edge eth0) │"
echo " └──────────────────────────────────────────────────┘"

check srv_public_web 203.0.113.10 "public web server (NAT test target)"
check ext_client 203.0.113.20 "external client (simulated attacker)"

echo ""
echo " ┌ Kernel Routing Tables ───────────────────────────┐"
echo " │ Each device should have only the connected route. │"
echo " │ No default route yet - that comes in step 3. │"
echo " └──────────────────────────────────────────────────┘"

echo ""

for host in srv_dns srv_web srv_db pc1_br1 pc2_br1 pc3_br1 \
pc1_br2 pc2_br2 pc3_br2 admin_pc srv_public_web ext_client; do
    echo " [$host]"
    show_routes "$host"
done

echo "======"
if [ "$FAIL" -eq 0 ]; then
    echo " ALL $PASS checks PASSED"
    echo " All hosts are correctly configured."
    echo " Next: bash scripts/03_routing/run.sh"
else
    echo " $PASS passed | $FAIL FAILED"
    echo " Fix the issues above before continuing."
fi
echo "======"

```

03_routing/run.sh

```

#!/bin/bash
# =====
# 03_routing/run.sh - Configure static routes

```

```
# After IP assignment, each device only knows its own subnet.
# This step teaches every device where to forward traffic
# destined outside its local subnet.
# =====

run_cmd() {
    local reason="$1"
    shift
    printf "      Reason : %s\n" "$reason"
    printf "      Command : %s\n" "$*"
    local output
    output=$(("$@" 2>&1)
    local exit_code=$?
    if [ -n "$output" ]; then
        echo "$output" | sed 's/^/      Output : /'
    else
        printf "      Output : (no output)\n"
    fi
    if [ "$exit_code" -eq 0 ]; then
        printf "      Status : done (exit 0)\n\n"
    else
        printf "      Status : FAILED (exit %d)\n\n" "$exit_code"
    fi
    return $exit_code
}

echo ""
echo "=====
echo "   STEP 3 – Static Routing
echo "=====
echo "   Devices only know their own subnet after IP assignment."
echo "   This step teaches every device where to forward traffic"
echo "   that is destined outside its local subnet."
echo "=====

echo ""
echo " ┌ Router Routes ────────────────────────────────────┐"
echo " │ Routers share net_servico, so they can use each   │"
echo " │ other as next-hops directly (no extra hops needed). │"
echo " └────────────────────────────────────────────────────┘"

# — r1_hq —————
echo ""
echo " [r1_hq] already knows: net_gerencia (.192/26), net_servico (.0/26)"
echo ""

run_cmd "r1_hq needs to forward Polo 1 traffic – sends it to r2_br1 as next-hop on
net_servico" \
    docker exec r1_hq ip route add 192.168.100.64/26 via 192.168.100.2

run_cmd "r1_hq needs to forward Polo 2 traffic – sends it to r3_br2 as next-hop on
net_servico" \
    docker exec r1_hq ip route add 192.168.100.128/26 via 192.168.100.3

run_cmd "r1_hq needs a path to the internet – any unknown destination goes to r4_edge" \
    docker exec r1_hq ip route add default via 192.168.100.4
```

```

# -- r2_br1 -----
echo " [r2_br1] already knows: net_polo1 (.64/26), net_servico (.0/26)"
echo ""

run_cmd "r2_br1 needs to forward Polo 2 traffic - sends it to r3_br2 as next-hop on
net_servico" \
    docker exec r2_br1 ip route add 192.168.100.128/26 via 192.168.100.3

run_cmd "r2_br1 needs to forward management traffic - sends it to r1_hq as next-hop on
net_servico" \
    docker exec r2_br1 ip route add 192.168.100.192/26 via 192.168.100.1

run_cmd "r2_br1 needs a path to the internet - any unknown destination goes to r4_edge" \
    docker exec r2_br1 ip route add default via 192.168.100.4

# -- r3_br2 -----
echo " [r3_br2] already knows: net_polo2 (.128/26), net_servico (.0/26)"
echo ""

run_cmd "r3_br2 needs to forward Polo 1 traffic - sends it to r2_br1 as next-hop on
net_servico" \
    docker exec r3_br2 ip route add 192.168.100.64/26 via 192.168.100.2

run_cmd "r3_br2 needs to forward management traffic - sends it to r1_hq as next-hop on
net_servico" \
    docker exec r3_br2 ip route add 192.168.100.192/26 via 192.168.100.1

run_cmd "r3_br2 needs a path to the internet - any unknown destination goes to r4_edge" \
    docker exec r3_br2 ip route add default via 192.168.100.4

# -- r4_edge -----
echo " [r4_edge] already knows: net_internet (203.0.113.0/24), net_servico (.0/26)"
echo "      No default needed - r4_edge IS the internet gateway"
echo ""

run_cmd "r4_edge needs this so NAT reply packets can find their way back to Polo 1 hosts" \
    docker exec r4_edge ip route add 192.168.100.64/26 via 192.168.100.2

run_cmd "r4_edge needs this so NAT reply packets can find their way back to Polo 2 hosts" \
    docker exec r4_edge ip route add 192.168.100.128/26 via 192.168.100.3

run_cmd "r4_edge needs this so NAT reply packets can find their way back to management
hosts" \
    docker exec r4_edge ip route add 192.168.100.192/26 via 192.168.100.1

# -- END HOSTS -----
echo ""
echo " ┌ Host Default Gateways ────────────────────────────────────────────────────┐"
echo " │ Each host sends unknown traffic to its local router. │"
echo " │ The router then handles forwarding from there. │"
echo " └────────────────────────────────────────────────────────────────────────────────┘"
echo ""

echo " [net_servico servers - default via r4_edge (192.168.100.4)]"
echo ""
run_cmd "srv_dns has no route to the internet yet - this adds it via r4_edge" \

```

```

    docker exec srv_dns ip route add default via 192.168.100.4
run_cmd "srv_web has no route to the internet yet - this adds it via r4_edge" \
    docker exec srv_web ip route add default via 192.168.100.4
run_cmd "srv_db has no route to the internet yet - this adds it via r4_edge" \
    docker exec srv_db ip route add default via 192.168.100.4

echo " [net_polo1 clients - default via r2_br1 (192.168.100.65)]"
echo ""
run_cmd "pc1_br1 only knows net_polo1 - this teaches it to send everything else to r2_br1" \
    docker exec pc1_br1 ip route add default via 192.168.100.65
run_cmd "pc2_br1 only knows net_polo1 - this teaches it to send everything else to r2_br1" \
    docker exec pc2_br1 ip route add default via 192.168.100.65
run_cmd "pc3_br1 only knows net_polo1 - this teaches it to send everything else to r2_br1" \
    docker exec pc3_br1 ip route add default via 192.168.100.65

echo " [net_polo2 clients - default via r3_br2 (192.168.100.129)]"
echo ""
run_cmd "pc1_br2 only knows net_polo2 - this teaches it to send everything else to r3_br2" \
    docker exec pc1_br2 ip route add default via 192.168.100.129
run_cmd "pc2_br2 only knows net_polo2 - this teaches it to send everything else to r3_br2" \
    docker exec pc2_br2 ip route add default via 192.168.100.129
run_cmd "pc3_br2 only knows net_polo2 - this teaches it to send everything else to r3_br2" \
    docker exec pc3_br2 ip route add default via 192.168.100.129

echo " [net_gerencia - default via r1_hq (192.168.100.193)]"
echo ""
run_cmd "admin_pc only knows net_gerencia - this teaches it to send everything else to r1_hq" \
    docker exec admin_pc ip route add default via 192.168.100.193

echo " [net_internet - default via r4_edge (203.0.113.1)]"
echo ""
run_cmd "srv_public_web needs a return path - sends unknown traffic back via r4_edge" \
    docker exec srv_public_web ip route add default via 203.0.113.1
run_cmd "ext_client needs a return path - sends unknown traffic back via r4_edge" \
    docker exec ext_client ip route add default via 203.0.113.1

echo "====="
echo " Routing configured."
echo " Run: bash scripts/03_routing/test.sh to verify"
echo "====="

```

03_routing/test.sh

```

#!/bin/bash
# =====
# 03_routing/test.sh - Verify static routing
# =====

```



```

PASS=0
FAIL=0

check_ping() {
    local from=$1
    local to_ip=$2
    local to_name=$3
    local description=$4

    printf "    Reason  : %s\n" "$description"
    printf "    Command : docker exec %s ping -c1 -W2 %s\n" "$from" "$to_ip"
    local output
    output=$(docker exec "$from" ping -c1 -W2 "$to_ip" 2>&1)
    local exit_code=$?
    local summary
    summary=$(echo "$output" | grep -E "packets transmitted" | head -1)
    printf "    Output  : %s\n" "$summary"
    if [ "$exit_code" -eq 0 ]; then
        printf "    Status  : [PASS] %s → %s (%s)\n\n" "$from" "$to_ip" "$to_name"
        PASS=$((PASS + 1))
    else
        printf "    Status  : [FAIL] %s → %s (%s)\n\n" "$from" "$to_ip" "$to_name"
        FAIL=$((FAIL + 1))
    fi
}

show_routes() {
    local device=$1
    printf "    Reason  : Confirm default gateway and static routes are present\n"
    printf "    Command : docker exec %s ip route\n" "$device"
    local output
    output=$(docker exec "$device" ip route 2>&1)
    echo "$output" | sed 's/^/    Output  : /'
    printf "    Status  : displayed\n\n"
}

echo ""
echo "=====
echo "  TEST 3 – Static Routing Verification
echo "=====

# — Level 1: Routing tables —————
echo ""
echo " ┌ Level 1 – Routing Tables ────────────────────────────────────┐"
echo " │ Each device should now have a default route in                │"
echo " │ addition to its kernel connected route.                       │"
echo " └────────────────────────────────────────────────────────────────┘"
echo ""

for device in r1_hq r2_br1 r3_br2 r4_edge srv_dns srv_web srv_db \
    pc1_br1 pc2_br1 pc3_br1 pc1_br2 pc2_br2 pc3_br2 \
    admin_pc srv_public_web ext_client; do
    echo "  [$device]"
    show_routes "$device"
done

```

```

# -- Level 2: Cross-subnet ping -----
echo ""
echo " ┌ Level 2 – Cross-Subnet Reachability ────────────┐"
echo " │ Packets must travel through routers to reach these │"
echo " │ destinations. Firewall not applied yet – all should │"
echo " │ pass. Note which ones will be blocked in step 5.     │"
echo " └──────────────────────────────────────────────────┘"
echo ""

echo " [Polo 1 → Servers]"
check_ping pc1_br1 192.168.100.10 "srv_dns" "Polo 1 reaching DNS across subnet via r2_br1"
check_ping pc1_br1 192.168.100.11 "srv_web" "Polo 1 reaching web server across subnet via r2_br1"
check_ping pc1_br1 192.168.100.12 "srv_db" "Polo 1 reaching DB – will be BLOCKED in step 5"

echo " [Polo 2 → Servers]"
check_ping pc1_br2 192.168.100.10 "srv_dns" "Polo 2 reaching DNS across subnet via r3_br2"
check_ping pc1_br2 192.168.100.11 "srv_web" "Polo 2 reaching web server across subnet via r3_br2"
check_ping pc1_br2 192.168.100.12 "srv_db" "Polo 2 reaching DB – will be BLOCKED in step 5"

echo " [Polo 1 ↔ Polo 2 – will be BLOCKED after firewall]"
check_ping pc1_br1 192.168.100.130 "pc1_br2" "Cross-branch – will be BLOCKED in step 5"
check_ping pc1_br2 192.168.100.66 "pc1_br1" "Cross-branch – will be BLOCKED in step 5"

echo " [Polos → Management – will be BLOCKED after firewall]"
check_ping pc1_br1 192.168.100.194 "admin_pc" "Polo 1 to management – will be BLOCKED in step 5"
check_ping pc1_br2 192.168.100.194 "admin_pc" "Polo 2 to management – will be BLOCKED in step 5"

# -- Level 3: Traceroute -----
echo ""
echo " ┌ Level 3 – Packet Path Trace ────────────┐"
echo " │ Shows every hop from pc1_br1 to srv_dns. │"
echo " │ Expected: pc1_br1 → r2_br1 (.65) → srv_dns (.10) │"
echo " └──────────────────────────────────────────────────┘"
echo ""
printf " Reason : Verify the exact routing path pc1_br1 → r2_br1 → srv_dns\n"
printf " Command : docker exec pc1_br1 traceroute -n -w1 192.168.100.10\n"
local_output=$(docker exec pc1_br1 traceroute -n -w1 192.168.100.10 2>&1)
echo "$local_output" | sed 's/^/ Output : /'
printf " Status : displayed\n"

echo ""
echo "=====
if [ "$FAIL" -eq 0 ]; then
    echo " ALL $PASS checks PASSED"
    echo " Routing is working correctly."
    echo " Next: bash scripts/04_nat/run.sh"
else
    echo " $PASS passed | $FAIL FAILED"
    echo " Fix the issues above before continuing."
fi
echo "=====

```

04_nat/run.sh

```
#!/bin/bash
# =====
# 04_nat/run.sh - Configure NAT on r4_edge
#
# Internal hosts use private IPs (192.168.100.x) that the
# internet cannot route back to. MASQUERADE NAT on r4_edge
# rewrites the source IP to its public-facing IP (203.0.113.1)
# so replies can find their way back.
# =====

run_cmd() {
    local reason="$1"
    shift
    printf "    Reason  : %s\n" "$reason"
    printf "    Command : %s\n" "$*"
    local output
    output=$(("$@" 2>&1)
    local exit_code=$?
    if [ -n "$output" ]; then
        echo "$output" | sed 's/^/    Output  : /'
    else
        printf "    Output  : (no output)\n"
    fi
    if [ "$exit_code" -eq 0 ]; then
        printf "    Status  : done (exit 0)\n\n"
    else
        printf "    Status  : FAILED (exit %d)\n\n" "$exit_code"
    fi
    return $exit_code
}

echo ""
echo "=====
echo "  STEP 4 - NAT Configuration"
echo "=====
echo "  Internal hosts use private IPs (192.168.100.x) that"
echo "  the internet cannot route back to. NAT on r4_edge"
echo "  translates those IPs to its own public IP (203.0.113.1)"
echo "  so replies can find their way back."
echo "=====

echo ""
echo "  ┌ [r4_edge] MASQUERADE NAT ────────────────────────────────────┐"
echo "  │ Applied on eth0 (internet-facing interface).                  │"
echo "  │                                                                 │"
echo "  │ Outbound: src 192.168.100.66 → rewritten to .113.1             │"
echo "  │ Inbound:  dst 203.0.113.1  → restored to .100.66              │"
echo "  │                                                                 │"
echo "  │ -t nat                : operate on the NAT table              │"
echo "  │ -A POSTROUTING        : apply just before packet leaves       │"
echo "  │ -o eth0               : only on packets leaving via eth0      │"
```

```

echo " | -j MASQUERADE : rewrite src IP to interface IP |"
echo " |_____|"
echo ""

R4_INTERNET=$(docker exec r4_edge ip addr show \
    | awk '/^[0-9]+:\/ { split($2,a,"@"); iface=a[1]; gsub(/:$/, "",iface) }
    /inet / && $2 ~ /203\.0\.113/ { print iface }')

printf "    Detected: net_internet interface on r4_edge = %s\n\n" "$R4_INTERNET"

run_cmd "Enable MASQUERADE on $R4_INTERNET so all internal hosts can reach the internet
using r4_edge's public IP" \
    docker exec r4_edge iptables -t nat -A POSTROUTING -o "$R4_INTERNET" -j MASQUERADE

echo "====="
echo "    NAT configured."
echo "    Run: bash scripts/04_nat/test.sh    to verify"
echo "====="

```

04_nat/test.sh

```

#!/bin/bash
# =====
# 04_nat/test.sh - Verify NAT is working
# =====

PASS=0
FAIL=0

check_ping() {
    local from=$1
    local to_ip=$2
    local to_name=$3
    local description=$4

    printf "    Reason : %s\n" "$description"
    printf "    Command : docker exec %s ping -c1 -W2 %s\n" "$from" "$to_ip"
    local output
    output=$(docker exec "$from" ping -c1 -W2 "$to_ip" 2>&1)
    local exit_code=$?
    local summary
    summary=$(echo "$output" | grep -E "packets transmitted" | head -1)
    printf "    Output : %s\n" "$summary"
    if [ "$exit_code" -eq 0 ]; then
        printf "    Status : [PASS] %s → %s (%s) reachable\n\n" "$from" "$to_ip" "$to_name"
        PASS=$((PASS + 1))
    else
        printf "    Status : [FAIL] %s → %s (%s) UNREACHABLE\n\n" "$from" "$to_ip"
"$to_name"
        FAIL=$((FAIL + 1))
    fi
}

echo ""
echo "====="

```

```

echo " TEST 4 - NAT Verification"
echo "=====

# -- Level 1: NAT rule present -----
echo ""
echo " ┌ Level 1 - NAT Rule on r4_edge ────────────┐"
echo " │ Confirms the MASQUERADE rule exists in the NAT table │"
echo " └──────────────────────────────────────────┘"
echo ""

printf " Reason : Inspect the full NAT POSTROUTING chain on r4_edge\n"
printf " Command : docker exec r4_edge iptables -t nat -L POSTROUTING -v\n"
nat_output=$(docker exec r4_edge iptables -t nat -L POSTROUTING -v 2>&1)
echo "$nat_output" | sed 's/^/ Output : /'
printf " Status : displayed\n\n"

printf " Reason : Confirm MASQUERADE keyword is present in the NAT table\n"
printf " Command : docker exec r4_edge iptables -t nat -L POSTROUTING | grep MASQUERADE\n"
grep_output=$(docker exec r4_edge iptables -t nat -L POSTROUTING 2>&1 | grep "MASQUERADE")
if [ -n "$grep_output" ]; then
    echo "$grep_output" | sed 's/^/ Output : /'
    printf " Status : [PASS] MASQUERADE rule is present on r4_edge\n\n"
    PASS=$((PASS + 1))
else
    printf " Output : (not found)\n"
    printf " Status : [FAIL] MASQUERADE rule MISSING - run 04_nat/run.sh first\n\n"
    FAIL=$((FAIL + 1))
fi

# -- Level 2: Internal hosts can reach internet -----
echo ""
echo " ┌ Level 2 - Internet Reachability via NAT ────────────┐"
echo " │ Packet leaves as 192.168.100.x, arrives as │"
echo " │ 203.0.113.1. A timeout means NAT is not working. │"
echo " └──────────────────────────────────────────┘"
echo ""

echo " [Polo 1 → Internet]"
check_ping pc1_br1 203.0.113.10 "srv_public_web" "pc1_br1 reaches internet via r2_br1 → r4_edge NAT"
check_ping pc2_br1 203.0.113.10 "srv_public_web" "pc2_br1 reaches internet via r2_br1 → r4_edge NAT"
check_ping pc3_br1 203.0.113.10 "srv_public_web" "pc3_br1 reaches internet via r2_br1 → r4_edge NAT"

echo " [Polo 2 → Internet]"
check_ping pc1_br2 203.0.113.10 "srv_public_web" "pc1_br2 reaches internet via r3_br2 → r4_edge NAT"
check_ping pc2_br2 203.0.113.10 "srv_public_web" "pc2_br2 reaches internet via r3_br2 → r4_edge NAT"
check_ping pc3_br2 203.0.113.10 "srv_public_web" "pc3_br2 reaches internet via r3_br2 → r4_edge NAT"

echo " [Management → Internet]"
check_ping admin_pc 203.0.113.10 "srv_public_web" "admin_pc reaches internet via r1_hq → r4_edge NAT"

```

```
# -- Level 3: Traceroute -----
echo ""
echo " ┌ Level 3 - Packet Path to Internet -----┐"
echo " │ Expected: pc1_br1 → r2_br1 (.65) → r4_edge → web │"
echo " │ After r4_edge the source IP becomes 203.0.113.1 │"
echo " └-----┘"
echo ""
printf " Reason : Trace the full path and observe NAT translation at r4_edge\n"
printf " Command : docker exec pc1_br1 traceroute -n -w1 203.0.113.10\n"
trace_output=$(docker exec pc1_br1 traceroute -n -w1 203.0.113.10 2>&1)
echo "$trace_output" | sed 's/^/ Output : /'
printf " Status : displayed\n"

echo ""
echo "=====
if [ "$FAIL" -eq 0 ]; then
    echo " ALL $PASS checks PASSED"
    echo " NAT is working correctly."
    echo " Next: bash scripts/05_firewall/run.sh"
else
    echo " $PASS passed | $FAIL FAILED"
    echo " Fix the issues above before continuing."
fi
echo "=====
```

05_firewall/run.sh

```
#!/bin/bash
# =====
# 05_firewall/run.sh - Apply iptables firewall rules
#
# Rules are applied to the FORWARD chain of each router.
# The FORWARD chain handles packets that PASS THROUGH a router
# (not destined to the router itself).
#
# Rule order matters - iptables processes top to bottom and
# stops at the first match. ESTABLISHED,RELATED always comes
# first so replies are never caught by a DROP rule below.
#
# Security policy summary:
# - Block Polo 1 ↔ Polo 2 (bidirectional)
# - Block Polos and gerencia from srv_db and srv_dns
# - Block Polos and gerencia from each other
# - Block internet from initiating NEW connections inward
# =====

run_cmd() {
    local reason="$1"
    shift
    printf " Reason : %s\n" "$reason"
    printf " Command : %s\n" "$*"
    local output
    output=$(("$@" 2>&1)
```

```

local exit_code=$?
if [ -n "$output" ]; then
    echo "$output" | sed 's/^/    Output : /'
else
    printf "    Output : (no output)\n"
fi
if [ "$exit_code" -eq 0 ]; then
    printf "    Status : done (exit 0)\n\n"
else
    printf "    Status : FAILED (exit %d)\n\n" "$exit_code"
fi
return $exit_code
}

echo ""
echo "=====
echo "  STEP 5 - Firewall Rules"
echo "=====
echo "  Rules go into the FORWARD chain - packets passing through"
echo "  the router, not destined to it."
echo "  ESTABLISHED,RELATED is always rule #1 so that replies to"
echo "  open connections are never accidentally dropped."
echo "=====

# -- r2_br1 - Polo 1 gateway -----
echo ""
echo "  ┌ [r2_br1] Polo 1 Gateway ────────────────────┐"
echo "  │ All outbound traffic from Polo 1 exits here first. │"
echo "  └────────────────────────────────────────────────┘"
echo ""

run_cmd "Rule #1 - must be first: allow reply packets for connections Polo 1 already opened
(stateful)" \
    docker exec r2_br1 iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

run_cmd "Rule #2 - block Polo 1 → Polo 2: bidirectional isolation requirement" \
    docker exec r2_br1 iptables -A FORWARD -s 192.168.100.64/26 -d 192.168.100.128/26 -j
DROP

run_cmd "Rule #3 - block Polo 1 → srv_db: database must not be reachable from branches" \
    docker exec r2_br1 iptables -A FORWARD -s 192.168.100.64/26 -d 192.168.100.12 -j DROP

run_cmd "Rule #4 - block Polo 1 → net_gerencia: management network must be isolated from
branches" \
    docker exec r2_br1 iptables -A FORWARD -s 192.168.100.64/26 -d 192.168.100.192/26 -j
DROP

run_cmd "Rule #5 - block Polo 1 → srv_dns: DNS server must not be reachable from branches"
\
    docker exec r2_br1 iptables -A FORWARD -s 192.168.100.64/26 -d 192.168.100.10 -j DROP

# -- r3_br2 - Polo 2 gateway -----
echo ""
echo "  ┌ [r3_br2] Polo 2 Gateway ────────────────────┐"
echo "  │ All outbound traffic from Polo 2 exits here first. │"
echo "  │ Mirror of r2_br1 rules for Polo 2 source range. │"
echo "  └────────────────────────────────────────────────┘"

```

```

echo ""

run_cmd "Rule #1 - must be first: allow reply packets for connections Polo 2 already opened (stateful)" \
    docker exec r3_br2 iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

run_cmd "Rule #2 - block Polo 2 → Polo 1: bidirectional isolation requirement" \
    docker exec r3_br2 iptables -A FORWARD -s 192.168.100.128/26 -d 192.168.100.64/26 -j DROP

run_cmd "Rule #3 - block Polo 2 → srv_db: database must not be reachable from branches" \
    docker exec r3_br2 iptables -A FORWARD -s 192.168.100.128/26 -d 192.168.100.12 -j DROP

run_cmd "Rule #4 - block Polo 2 → net_gerencia: management network must be isolated from branches" \
    docker exec r3_br2 iptables -A FORWARD -s 192.168.100.128/26 -d 192.168.100.192/26 -j DROP

run_cmd "Rule #5 - block Polo 2 → srv_dns: DNS server must not be reachable from branches" \
    docker exec r3_br2 iptables -A FORWARD -s 192.168.100.128/26 -d 192.168.100.10 -j DROP

# --- r1_hq - Gerencia gateway ---
echo ""
echo " ┌ [r1_hq] Management Gateway ────────────┐ "
echo " │ All traffic leaving net_gerencia exits here. │ "
echo " │ Blocks admin from reaching branches and database. │ "
echo " └──────────────────────────────────────────┘ "
echo ""

run_cmd "Rule #1 - must be first: allow reply packets for connections admin already opened (stateful)" \
    docker exec r1_hq iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

run_cmd "Rule #2 - block gerencia → Polo 1: management must not access branch clients" \
    docker exec r1_hq iptables -A FORWARD -s 192.168.100.192/26 -d 192.168.100.64/26 -j DROP

run_cmd "Rule #3 - block gerencia → Polo 2: management must not access branch clients" \
    docker exec r1_hq iptables -A FORWARD -s 192.168.100.192/26 -d 192.168.100.128/26 -j DROP

run_cmd "Rule #4 - block gerencia → srv_db: database must not be reachable from management either" \
    docker exec r1_hq iptables -A FORWARD -s 192.168.100.192/26 -d 192.168.100.12 -j DROP

run_cmd "Rule #5 - block gerencia → srv_dns: DNS server must not be reachable from management either" \
    docker exec r1_hq iptables -A FORWARD -s 192.168.100.192/26 -d 192.168.100.10 -j DROP

# --- r4_edge - Stateful internet firewall ---
echo ""
echo " ┌ [r4_edge] Stateful Internet Firewall ────────────┐ "
echo " │ Blocks internet from initiating connections inward. │ "
echo " │ Internal hosts can still reach the internet - their │ "
echo " │ replies come back in via ESTABLISHED,RELATED. │ "
echo " └──────────────────────────────────────────┘ "
echo ""

```



```

R4_INTERNET=$(docker exec r4_edge ip addr show \
    | awk '/^[0-9]+:/ { split($2,a,"@"); iface=a[1]; gsub(/:$/, "",iface) }
    /inet / && $2 ~ /203\.0\.113/ { print iface }')

printf "    Detected: net_internet interface on r4_edge = %s\n\n" "$R4_INTERNET"

run_cmd "Rule #1 - must be first: allow replies to connections that internal hosts already
opened" \
    docker exec r4_edge iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

run_cmd "Rule #2 - block internet → internal: drop any NEW connection arriving from
$R4_INTERNET (internet side)" \
    docker exec r4_edge iptables -A FORWARD -i "$R4_INTERNET" -m state --state NEW -j DROP

echo "====="
echo "    Firewall rules applied."
echo "    Run: bash scripts/05_firewall/test.sh    to verify"
echo "====="

```

05_firewall/test.sh

```

#!/bin/bash
# =====
# 05_firewall/test.sh - Verify all firewall rules
# =====

PASS=0
FAIL=0

expect_pass() {
    local from=$1
    local to_ip=$2
    local to_name=$3
    local description=$4

    printf "    Reason   : %s\n" "$description"
    printf "    Command  : docker exec %s ping -c1 -W2 %s\n" "$from" "$to_ip"
    local output
    output=$(docker exec "$from" ping -c1 -W2 "$to_ip" 2>&1)
    local exit_code=$?
    local summary
    summary=$(echo "$output" | grep -E "packets transmitted" | head -1)
    printf "    Output   : %s\n" "$summary"
    if [ "$exit_code" -eq 0 ]; then
        printf "    Status   : [PASS] %s → %s (%s) reachable as expected\n\n" "$from"
"$to_ip" "$to_name"
        PASS=$((PASS + 1))
    else
        printf "    Status   : [FAIL] %s → %s (%s) UNREACHABLE - should be allowed\n\n"
"$from" "$to_ip" "$to_name"
        FAIL=$((FAIL + 1))
    fi
}

```

```

expect_block() {
    local from=$1
    local to_ip=$2
    local to_name=$3
    local description=$4

    printf "    Reason : %s\n" "$description"
    printf "    Command : docker exec %s ping -c1 -W2 %s\n" "$from" "$to_ip"
    local output
    output=$(docker exec "$from" ping -c1 -W2 "$to_ip" 2>&1)
    local exit_code=$?
    local summary
    summary=$(echo "$output" | grep -E "packets transmitted" | head -1)
    printf "    Output : %s\n" "$summary"
    if [ "$exit_code" -eq 0 ]; then
        printf "    Status : [FAIL] %s → %s (%s) reachable - should be BLOCKED\n\n" "$from"
"$to_ip" "$to_name"
        FAIL=$((FAIL + 1))
    else
        printf "    Status : [PASS] %s → %s (%s) blocked as expected\n\n" "$from" "$to_ip"
"$to_name"
        PASS=$((PASS + 1))
    fi
}

echo ""
echo "=====
echo "  TEST 5 - Firewall Verification
echo "=====

# -- Active rules dump -----
echo ""
echo "  ┌ Active FORWARD Rules ────────────────────────────────────────────────────┐"
echo "  │ Shows all iptables rules currently on each router. │"
echo "  └───────────────────────────────────────────────────────────────────────────┘"
echo ""

for router in r1_hq r2_br1 r3_br2 r4_edge; do
    printf "    Reason : Inspect all FORWARD rules on %s\n" "$router"
    printf "    Command : docker exec %s iptables -L FORWARD -v\n" "$router"
    fw_output=$(docker exec "$router" iptables -L FORWARD -v 2>&1)
    echo "$fw_output" | sed 's/^/    Output : /'
    printf "    Status : displayed\n\n"
done

# -- Must PASS -----
echo ""
echo "  ┌ Connectivity - Must PASS ───────────────────────────────────────────────────┐"
echo "  │ These connections must still work after the firewall. │"
echo "  └───────────────────────────────────────────────────────────────────────────┘"
echo ""

echo "  [Polo 1 → Allowed destinations]"
expect_pass pc1_br1 192.168.100.11 "srv_web" "Polo 1 must reach web server"
expect_pass pc1_br1 203.0.113.10 "srv_pub" "Polo 1 must reach internet via NAT"

echo "  [Polo 2 → Allowed destinations]"

```

```

expect_pass pc1_br2 192.168.100.11 "srv_web" "Polo 2 must reach web server"
expect_pass pc1_br2 203.0.113.10 "srv_pub" "Polo 2 must reach internet via NAT"

# -- Must BLOCK -----
echo ""
echo "  ┌ Isolation - Must BLOCK -----┐"
echo "  │   These connections must be dropped by the firewall.   │"
echo "  └-----┘"
echo ""

echo "  [Polo 1 ↔ Polo 2 - bidirectional block]"
expect_block pc1_br1 192.168.100.130 "pc1_br2" "r2_br1 drops src=polo1 dst=polo2"
expect_block pc1_br2 192.168.100.66 "pc1_br1" "r3_br2 drops src=polo2 dst=polo1"

echo "  [Polos → srv_db]"
expect_block pc1_br1 192.168.100.12 "srv_db" "r2_br1 drops src=polo1 dst=srv_db"
expect_block pc1_br2 192.168.100.12 "srv_db" "r3_br2 drops src=polo2 dst=srv_db"

echo "  [Polos → srv_dns]"
expect_block pc1_br1 192.168.100.10 "srv_dns" "r2_br1 drops src=polo1 dst=srv_dns"
expect_block pc1_br2 192.168.100.10 "srv_dns" "r3_br2 drops src=polo2 dst=srv_dns"

echo "  [Polos → net_gerencia]"
expect_block pc1_br1 192.168.100.194 "admin_pc" "r2_br1 drops src=polo1 dst=gerencia"
expect_block pc1_br2 192.168.100.194 "admin_pc" "r3_br2 drops src=polo2 dst=gerencia"

echo "  [gerencia → Polos]"
expect_block admin_pc 192.168.100.66 "pc1_br1" "r1_hq drops src=gerencia dst=polo1"
expect_block admin_pc 192.168.100.130 "pc1_br2" "r1_hq drops src=gerencia dst=polo2"

echo "  [gerencia → srv_db]"
expect_block admin_pc 192.168.100.12 "srv_db" "r1_hq drops src=gerencia dst=srv_db"

echo "  [gerencia → srv_dns]"
expect_block admin_pc 192.168.100.10 "srv_dns" "r1_hq drops src=gerencia dst=srv_dns"

echo "  [Internet → internal (stateful)]"
expect_block ext_client 192.168.100.66 "pc1_br1" "r4_edge drops NEW connection from eth0
to Polo 1"
expect_block ext_client 192.168.100.130 "pc1_br2" "r4_edge drops NEW connection from eth0
to Polo 2"
expect_block ext_client 192.168.100.194 "admin_pc" "r4_edge drops NEW connection from eth0
to management"

echo ""
echo "=====
if [ "$FAIL" -eq 0 ]; then
    echo "  ALL $PASS checks PASSED"
    echo "  Security policy is correctly enforced."
else
    echo "  $PASS passed | $FAIL FAILED"
    echo "  Fix the issues above before submitting."
fi
echo "=====

```